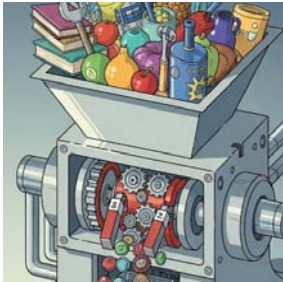


B 4.1.6 Core principles of ADTs - Python (HL Only)



Discover the power behind Python's dict and set in this guide to Abstract Data Types and hash tables. Learn how hashing functions, collision resolution, and load factors enable lightning-fast data operations. This chapter breaks down the core principles of the Map and Set ADTs, showing how these theoretical concepts are implemented to provide Python with its most efficient data structures.

Introduction

Welcome to this section on one of the most important concepts in computer science: the hash table. Have you ever wondered how a Python dictionary or set can find an item almost instantly, even among millions of entries? The answer lies in the clever mechanics of the hash table.

This chapter will explain the high-level concept of Abstract Data Types (ADTs), dive into the underlying mechanics of how hash tables work, and show how this powers Python's essential `dict` and `set` types.

High-Level Description of Data Structures

An **Abstract Data Type (ADT)** is a theoretical blueprint for a data structure. It defines *what* a data structure should do by specifying a set of operations, but it says nothing about *how* it should be implemented.

Think of it like the controls of a car:

- **ADT (The Interface):** You have a steering wheel, an accelerator, and a brake. You know their purpose and the operations they perform (`turn`, `go_faster`, `stop`).
- **Concrete Data Structure (The Implementation):** This is the engine, the hydraulics, the axles. It's the complex machinery that actually makes the operations work.

You don't need to be a mechanic to drive a car. In the same way, you can use a data structure without knowing its internal code, as long as you understand its ADT (its purpose and operations).

The two key ADTs we'll focus on are the **Map** (also called a Dictionary) and the **Set**.

Map ADT:

- **Purpose:** To store a collection of **key-value pairs**.
- **Operations:** `insert(key, value)`, `get(key)`, `remove(key)`.

Set ADT:

- **Purpose:** To store a collection of **unique, unordered** elements.
- **Operations:** `add(element)`, `remove(element)`, `contains(element)`.

The Underlying Mechanics of Hash Tables

The **hash table** is the brilliant data structure that commonly implements both the Map and Set ADTs, providing them with their incredible speed.

The secret is that a hash table is just a standard **array** (or a Python list) under the hood. The challenge it solves is how to turn a key (which could be a string like "apple") into an array index (an integer like 5). This is done through a two-step process.

Hashing Functions

A **hashing function** is a process that converts a key into an array index.

1. **Generate a Hash Code:** First, the key is passed to a function that converts it into a large integer. This integer is called the **hash code**. A good hashing function is **deterministic**, meaning the same key will always produce the same hash code.
2. **Map to an Index:** To make sure this large integer fits within our array, we use the **modulo operator (%)**.

```
index = hash_code % array_size
```

For example, if the key "apple" has a hash code of 133742 and our array has 10 slots, the index would be $133742 \% 10 = 2$. The data for "apple" would be stored at index 2 of the array. This calculation is almost instantaneous, giving us an **O(1)** lookup time.

Collision Resolution Strategies

What happens if two different keys produce the same index? This is called a **hash collision**.

For example, $\text{hash}(\text{"cat"}) \% 5 = 3$ and $\text{hash}(\text{"dog"}) \% 5 = 3$. They can't both go in slot 3! All hash tables must have a strategy to resolve this.

- **Linear Probing:** The simplest strategy. If your slot is taken, just try the next one (`index + 1`). If that one is taken, try the next one after that (`index + 2`), and so on, wrapping around to the start of the array if you reach the end.
- **Chaining:** A more robust strategy. Instead of each array slot holding a single item, it holds a **list** of items. When a collision occurs, the new item is simply added to the list at that index.

Load Factor and Rehashing

The **load factor** tells us how full the hash table is. $\text{Load Factor} = (\text{Number of Items}) / (\text{Number of Table Slots})$

As the load factor increases, collisions become more frequent and performance drops. To fix this, when the load factor reaches a certain threshold (e.g., ~ 0.75), the hash table **rehashes**:

1. A new, larger array is created (often double the size).
2. All items from the old table are re-inserted into the new one. Their indices are recalculated because the array size has changed.

This is a slow, one-time operation that ensures future lookups, insertions, and deletions remain fast.

The Underlying Mechanics of Sets

How does a **set** use a hash table? A set is just a collection of unique keys without any associated values.

Under the hood, a Python **set** is implemented as a hash table where the elements of the set are stored as the **keys**. The "values" are ignored.

This is why checking for membership is so fast: `if "apple" in my_set:`

Python simply calculates the hash of "apple" to find the index where it *should* be and checks that one spot (or a very short chain). This is an $O(1)$ operation, far faster than searching a list.

dict and **set** in Python

dict (Dictionary): This is Python's built-in implementation of the Map ADT using a hash table. It is used to store **key-value pairs**.

```
# Key: String, Value: Integer
student_scores = {"Maria": 92, "Chen": 88}

# Accessing a value by key
print(student_scores["Maria"]) # Output: 92
```

set: This is Python's built-in implementation of the Set ADT, also using a hash table. It is used to store **unique, unordered elements**.

- ```
A set of unique tags
tags = {"python", "data", "code", "python"}

print(tags) # Output: {'python', 'code', 'data'}

Fast membership testing
print("data" in tags) # True
```

## Now test yourself!

1

What is the primary role of an Abstract Data Type (ADT)?

- A. ☒ To define the logical behaviour and operations of a data structure, hiding the implementation details.
- B. ☐ To define the syntax of a programming language.
- C. ☐ To store data directly in hardware.
- D. ☐ To provide the most efficient code for a data structure.

2

What is the main purpose of a hashing function?

- A. ☐ To encrypt a key for security.
- B. ☐ To sort the keys in a hash table.
- C. ☐ To calculate the load factor.
- D. ☒ To convert a key into an array index.

3

A hash collision occurs when...

- A. ☐ The hash table needs to be rehashed.
- B. ☐ The load factor is greater than 1.
- C. ☒ Two different keys map to the same index.
- D. ☐ A key is not found in the table.

4

In the chaining method of collision resolution, what is stored at each array index?

- A. ☒ A list (or linked list) of key-value pairs.
- B. ☐ The load factor of that index.
- C. ☐ A reference to the next available empty slot.
- D. ☐ A single key-value pair.

5

What is the "load factor" a measure of?

- A. ☒ How full the hash table is (items divided by slots).
- B. ☐ How much memory the hash table is using.
- C. ☐ The number of collisions that have occurred.
- D. ☐ How fast the hash function is.

6

What happens during rehashing?

- A. ☐ All keys are sorted alphabetically.
- B. ☐ Collided items are deleted from the table.
- C. ☒ A new, larger array is created and all items are re-inserted.
- D. ☐ The hashing function is replaced with a better one.

7

How is a Python set implemented internally?

- A. ☒ As a hash table where only the keys are used.
- B. ☐ As a sorted list.
- C. ☐ As a stack.
- D. ☐ As a binary search tree.

8

What is the ideal time complexity for a lookup in a hash table?

- A. ☐  $O(n)$
- B. ☐  $O(\log n)$
- C. ☐  $O(n^2)$

D. ☒  $O(1)$

9

Which of these is a key-value pair?

A. ☒ "username": "test\_user"

B. ☐ [10, 20, 30]

C. ☐ (1, 2, 3)

D. ☐ {"apple"}

10

Which data structure in Python is the concrete implementation of a hash table for key-value pairs?

A. ☐ tuple

B. ☐ list

C. ☐ set

D. ☒ dict

---

*All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.*

---

 **Report a problem**